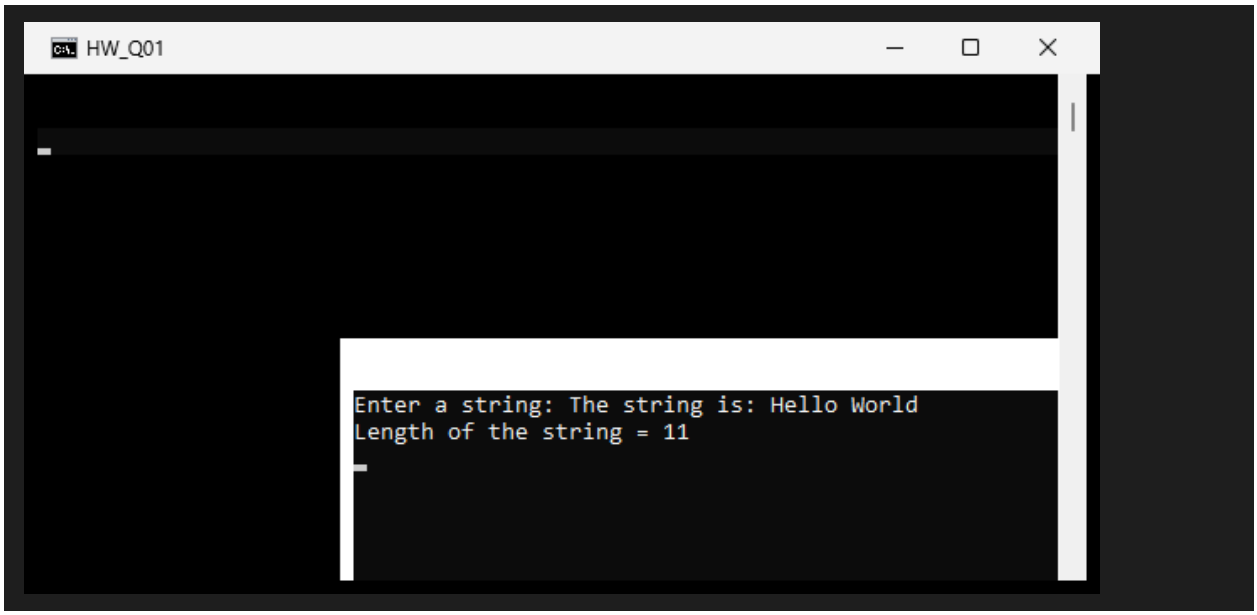
# Amit_Lab4 - Day 4 Assignment

*C Programming - Strings*

## Question 1: String length using strlen()

```c
/* Q1. Read a string from the user and display its length using strlen(). */
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    printf("The string is: %s\n", str);
    printf("Length of the string = %d\n", (int)strlen(str));

    return 0;
}
```

**Output:**




---

## Question 2: Copy a string using strcpy()

```c
/* Q2. Input a string and copy it into another string using strcpy().
       Display both the original and copied strings. */
#include <stdio.h>
#include <string.h>

int main()
{
    char original[100], copied[100];
```

```
    printf("Enter a string: ");
    fgets(original, sizeof(original), stdin);
    original[strcspn(original, "\n")] = '\0';

    strcpy(copied, original);

    printf("Original string: %s\n", original);
    printf("Copied string  : %s\n", copied);

    return 0;
}
```

**Output:**

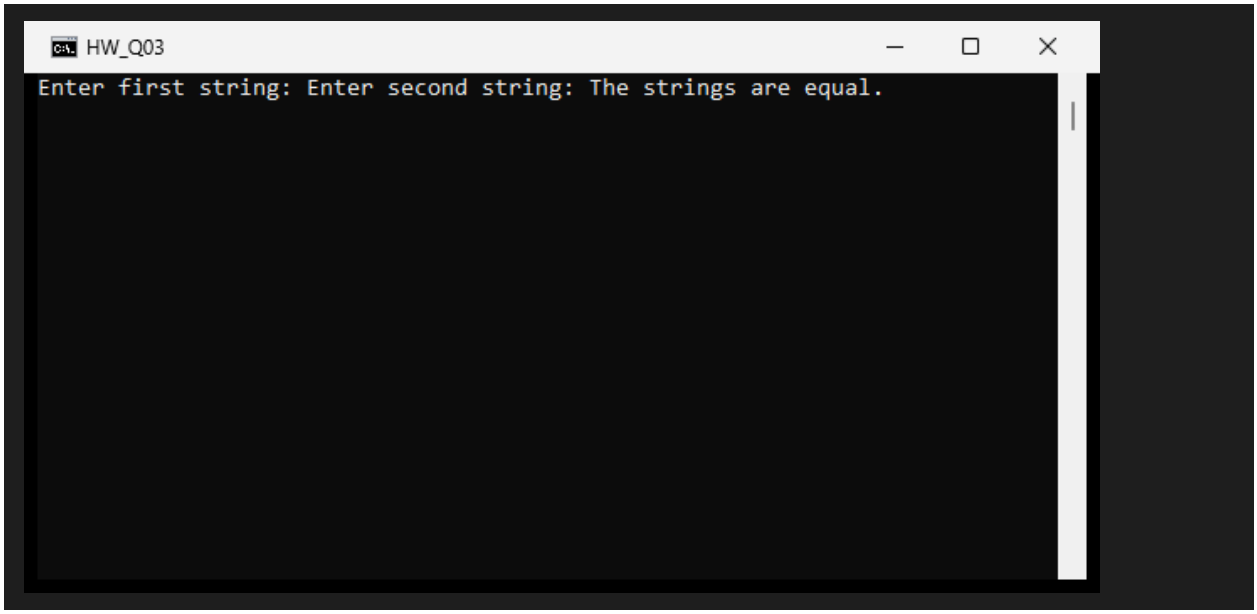

---

## Question 3: Compare two strings using strcmp()

```
/* Q3. Input two strings and compare them using strcmp().
       Display whether they are equal or not. */
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[100], s2[100];

    printf("Enter first string: ");
    fgets(s1, sizeof(s1), stdin);
    s1[strcspn(s1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(s2, sizeof(s2), stdin);
    s2[strcspn(s2, "\n")] = '\0';

    if (strcmp(s1, s2) == 0)
        printf("The strings are equal.\n");
    else
        printf("The strings are not equal.\n");
```

```
    return 0;
}
```

**Output:**

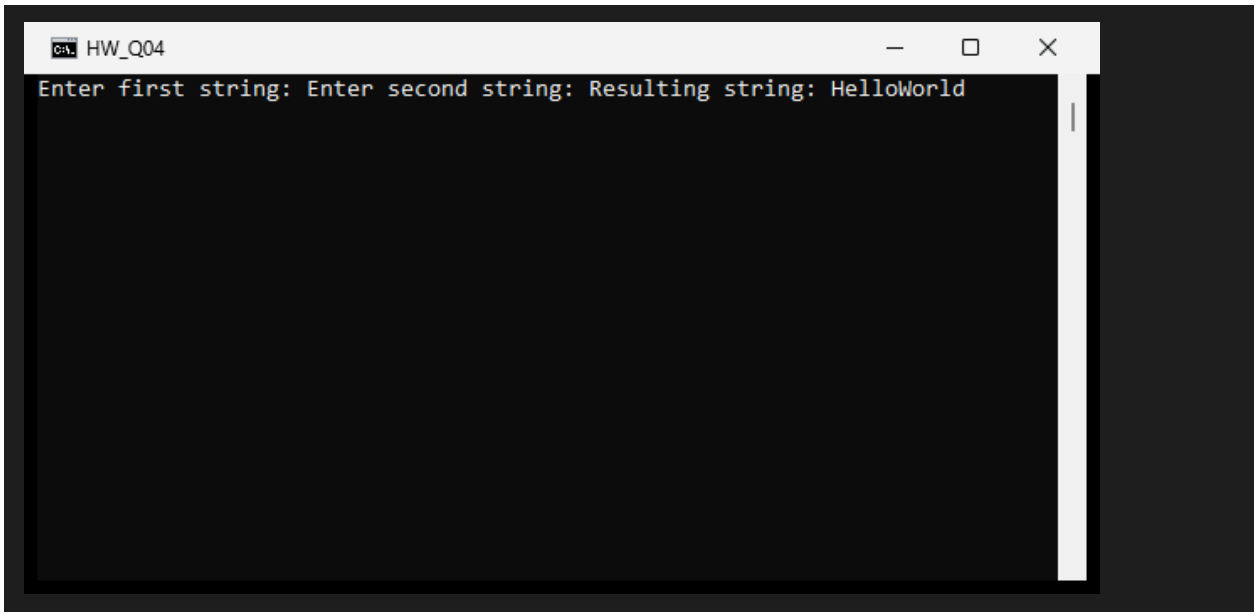



---

## Question 4: Join two strings using strcat()

```
/* Q4. Input two strings and join them using strcat().
       Display the resulting string. */
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[200], s2[100];

    printf("Enter first string: ");
    fgets(s1, sizeof(s1), stdin);
    s1[strcspn(s1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(s2, sizeof(s2), stdin);
    s2[strcspn(s2, "\n")] = '\0';

    strcat(s1, s2);

    printf("Resulting string: %s\n", s1);

    return 0;
}
```

**Output:**

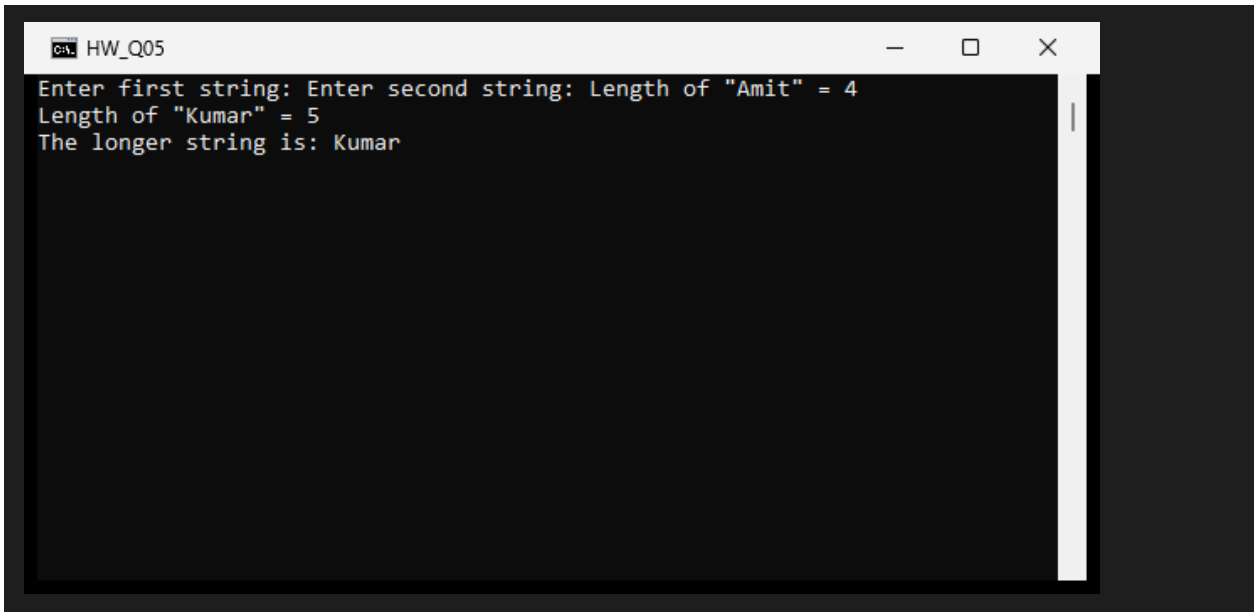



---

## Question 5: Display the longer of two strings using strlen()

```
/* Q5. Input two strings. Compare their lengths using strlen()
       and display the longer string. */
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[100], s2[100];

    printf("Enter first string: ");
    fgets(s1, sizeof(s1), stdin);
    s1[strcspn(s1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(s2, sizeof(s2), stdin);
    s2[strcspn(s2, "\n")] = '\0';

    printf("Length of \"%s\" = %d\n", s1, (int)strlen(s1));
    printf("Length of \"%s\" = %d\n", s2, (int)strlen(s2));

    if (strlen(s1) > strlen(s2))
        printf("The longer string is: %s\n", s1);
    else if (strlen(s1) < strlen(s2))
        printf("The longer string is: %s\n", s2);
    else
        printf("Both strings have the same length.\n");

    return 0;
}
```

**Output:**

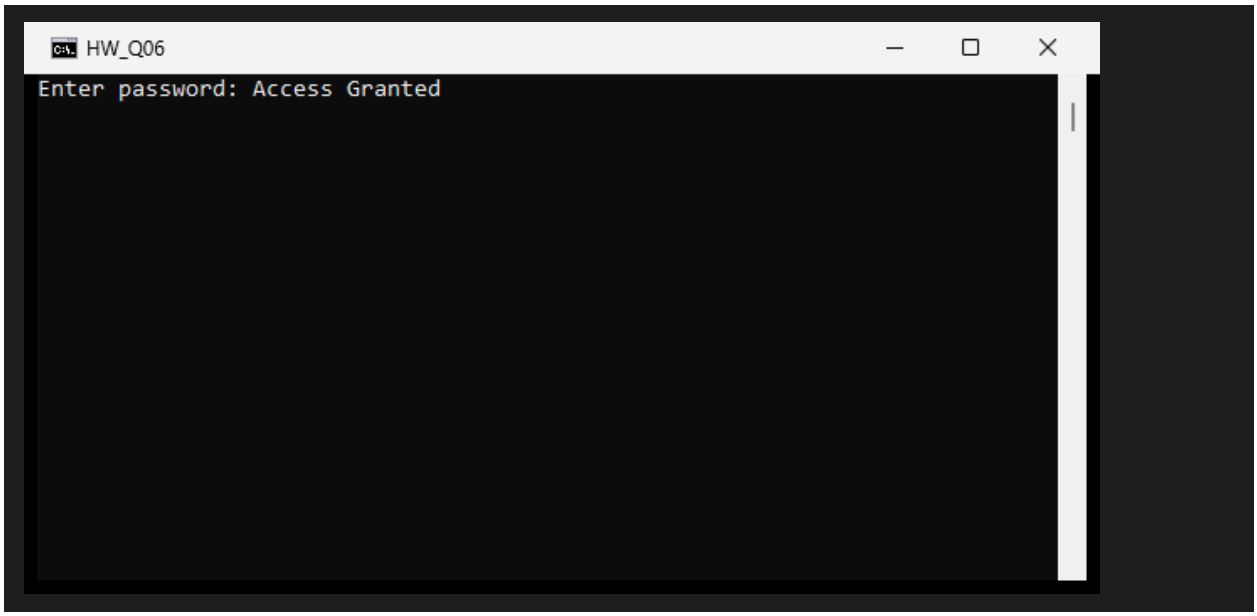



---

## Question 6: Password check using strcmp()

```
/* Q6. Assume the correct password is "admin123". Read a password from the user
       and compare it using strcmp(). Display "Access Granted" or "Access Denied". */
#include <stdio.h>
#include <string.h>

int main()
{
    char password[100];

    printf("Enter password: ");
    fgets(password, sizeof(password), stdin);
    password[strcspn(password, "\n")] = '\0';

    if (strcmp(password, "admin123") == 0)
        printf("Access Granted\n");
    else
        printf("Access Denied\n");

    return 0;
}
```

**Output:**

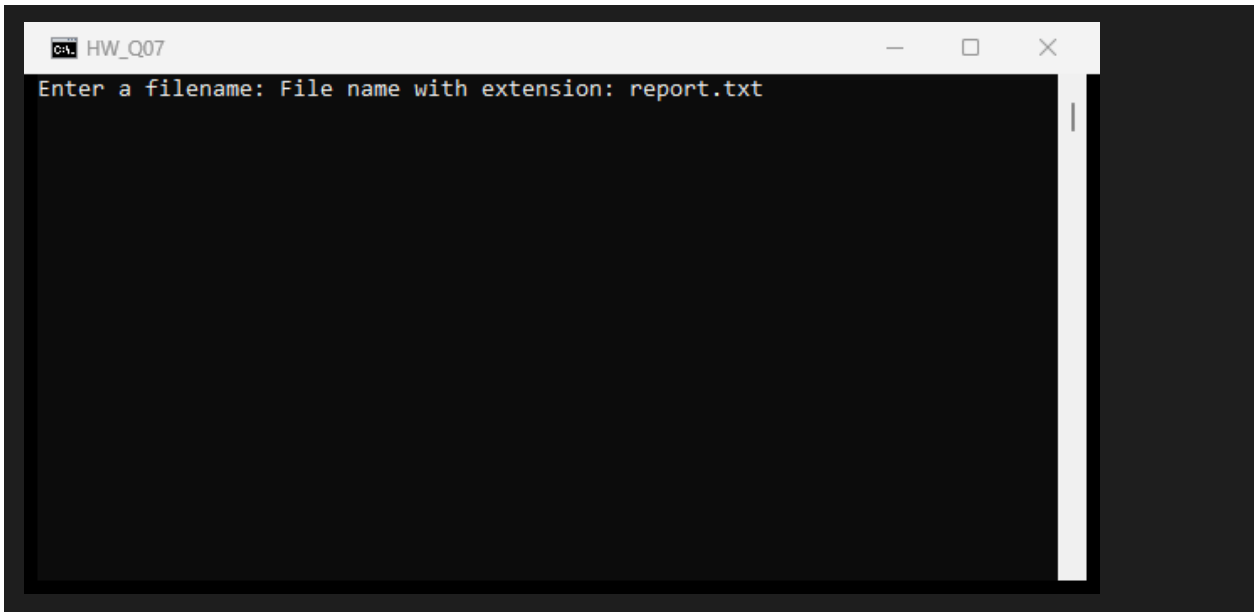



---

## Question 7: Append .txt extension to a filename using strcat()

```c
/* Q7. Input a filename and append the extension .txt using strcat(). */
#include <stdio.h>
#include <string.h>

int main()
{
    char filename[100];

    printf("Enter a filename: ");
    fgets(filename, sizeof(filename), stdin);
    filename[strcspn(filename, "\n")] = '\0';

    strcat(filename, ".txt");

    printf("File name with extension: %s\n", filename);

    return 0;
}
```

**Output:**

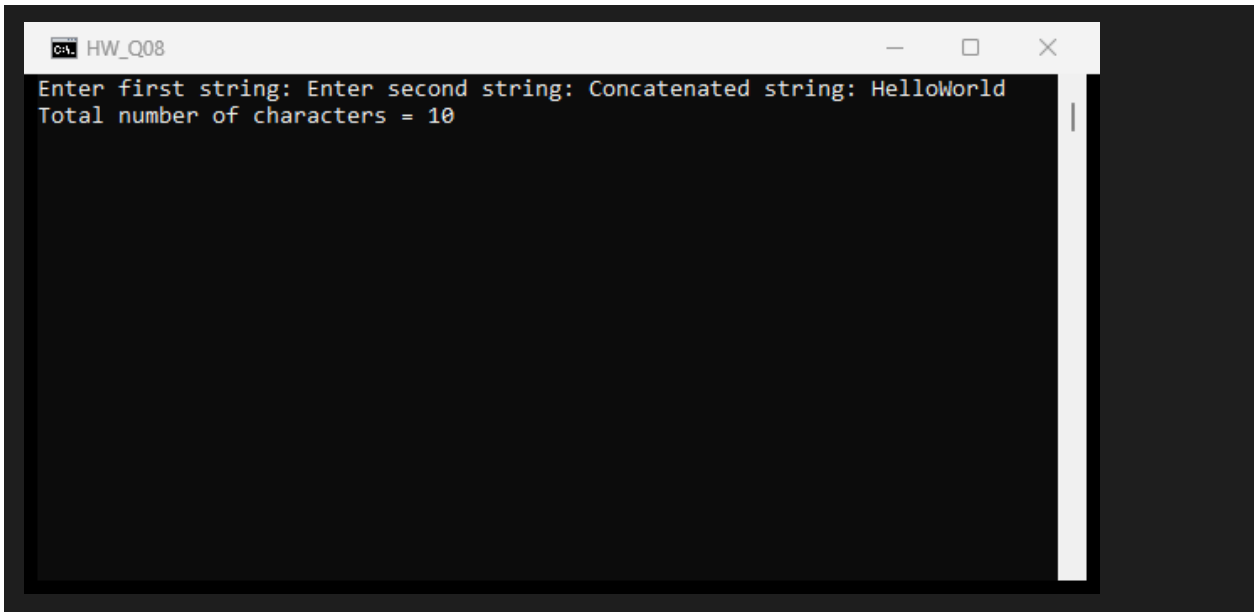



---

## Question 8: Concatenate two strings and display total character count

```c
/* Q8. Input two strings, concatenate them using strcat(), and display
       the concatenated string and the total number of characters using strlen(). */
#include <stdio.h>
#include <string.h>

int main()
{
    char s1[200], s2[100];

    printf("Enter first string: ");
    fgets(s1, sizeof(s1), stdin);
    s1[strcspn(s1, "\n")] = '\0';

    printf("Enter second string: ");
    fgets(s2, sizeof(s2), stdin);
    s2[strcspn(s2, "\n")] = '\0';

    strcat(s1, s2);

    printf("Concatenated string: %s\n", s1);
    printf("Total number of characters = %d\n", (int)strlen(s1));

    return 0;
}
```

**Output:**

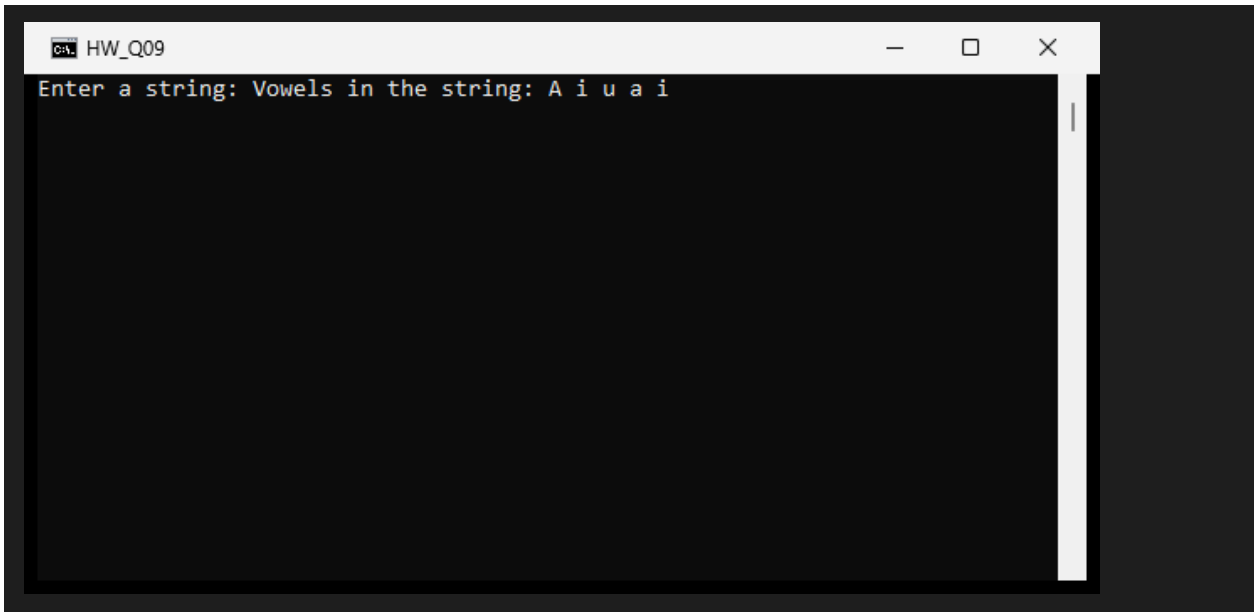



---

## Question 9: Print all vowels present in a string

```
/* Q9. Accept a string. Define a vowel array containing lowercase and uppercase
       vowels. Iterate through each character and compare it against the vowel
       array. Print each character from the input string that is a vowel,
       followed by a space. */
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100];
    char vowels[] = {'a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'};
    int i, j;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    printf("Vowels in the string: ");
    for (i = 0; str[i] != '\0'; i++)
    {
        for (j = 0; j < 10; j++)
        {
            if (str[i] == vowels[j])
            {
                printf("%c ", str[i]);
                break;
            }
        }
    }
    printf("\n");

    return 0;
}
```

**Output:**

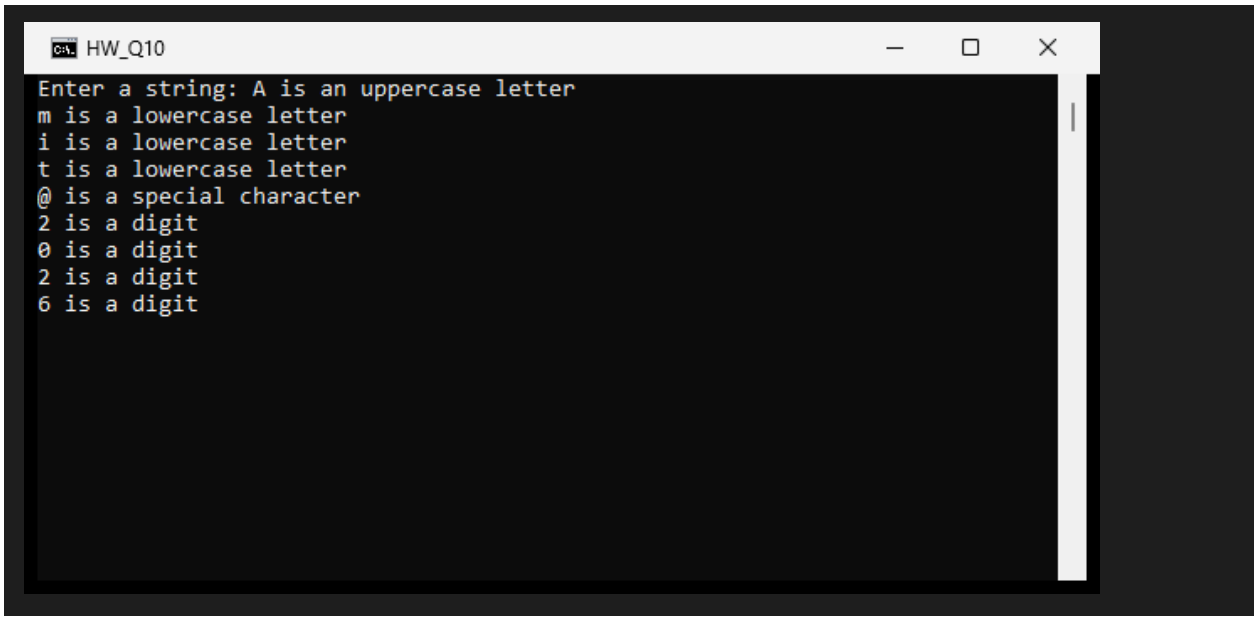



---

## Question 10: Identify each character as lowercase, uppercase, digit, or special character

```c
/* Q10. Accept a string, then print each character one by one while also
        identifying whether it is a lowercase letter, an uppercase letter,
        a digit, or a special character. */
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main()
{
    char str[100];
    int i;

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    str[strcspn(str, "\n")] = '\0';

    for (i = 0; str[i] != '\0'; i++)
    {
        if (islower(str[i]))
            printf("%c is a lowercase letter\n", str[i]);
        else if (isupper(str[i]))
            printf("%c is an uppercase letter\n", str[i]);
        else if (isdigit(str[i]))
            printf("%c is a digit\n", str[i]);
        else
            printf("%c is a special character\n", str[i]);
    }

    return 0;
}
```

**Output:**

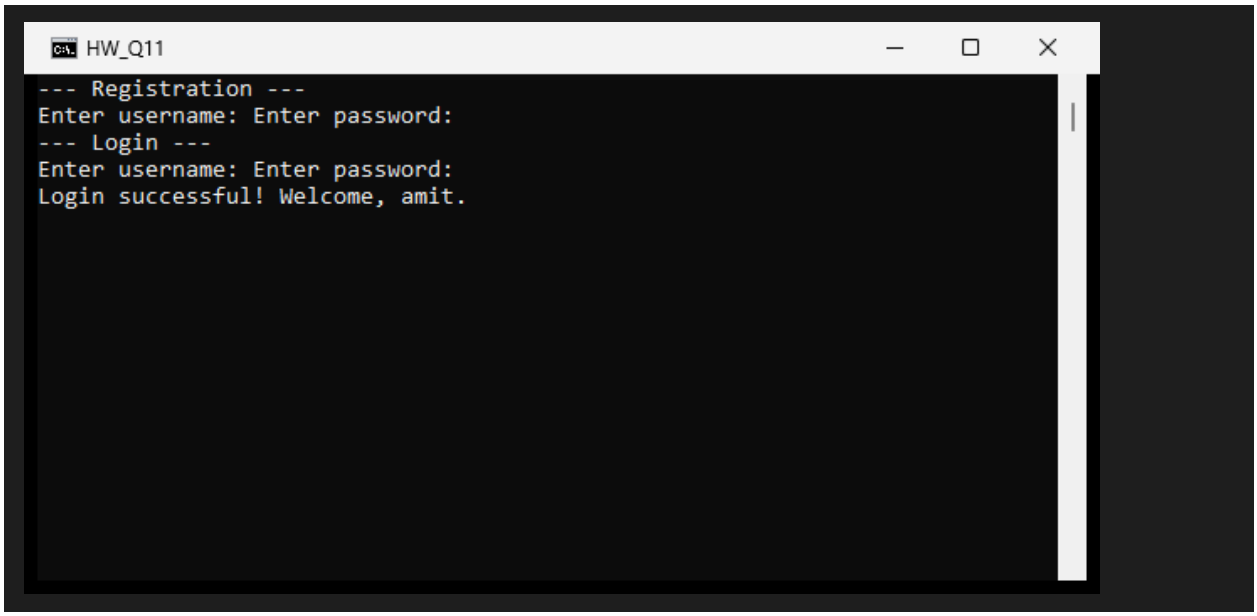



---

## Question 11: Simple C Login System

```
/* Q11. Simple C Login System:
       - Register a user: prompt to set a username and a password.
       - Login: prompt to enter username and password.
       - Validate credentials against the stored ones.
       - Display success or failure message. */
#include <stdio.h>
#include <string.h>

int main()
{
    char regUser[50], regPass[50];
    char loginUser[50], loginPass[50];

    printf("--- Registration ---\n");
    printf("Enter username: ");
    scanf("%49s", regUser);
    printf("Enter password: ");
    scanf("%49s", regPass);

    printf("\n--- Login ---\n");
    printf("Enter username: ");
    scanf("%49s", loginUser);
    printf("Enter password: ");
    scanf("%49s", loginPass);

    if (strcmp(regUser, loginUser) == 0 && strcmp(regPass, loginPass) == 0)
        printf("\nLogin successful! Welcome, %s.\n", loginUser);
    else
        printf("\nLogin failed! Invalid username or password.\n");

    return 0;
}
```

**Output:**



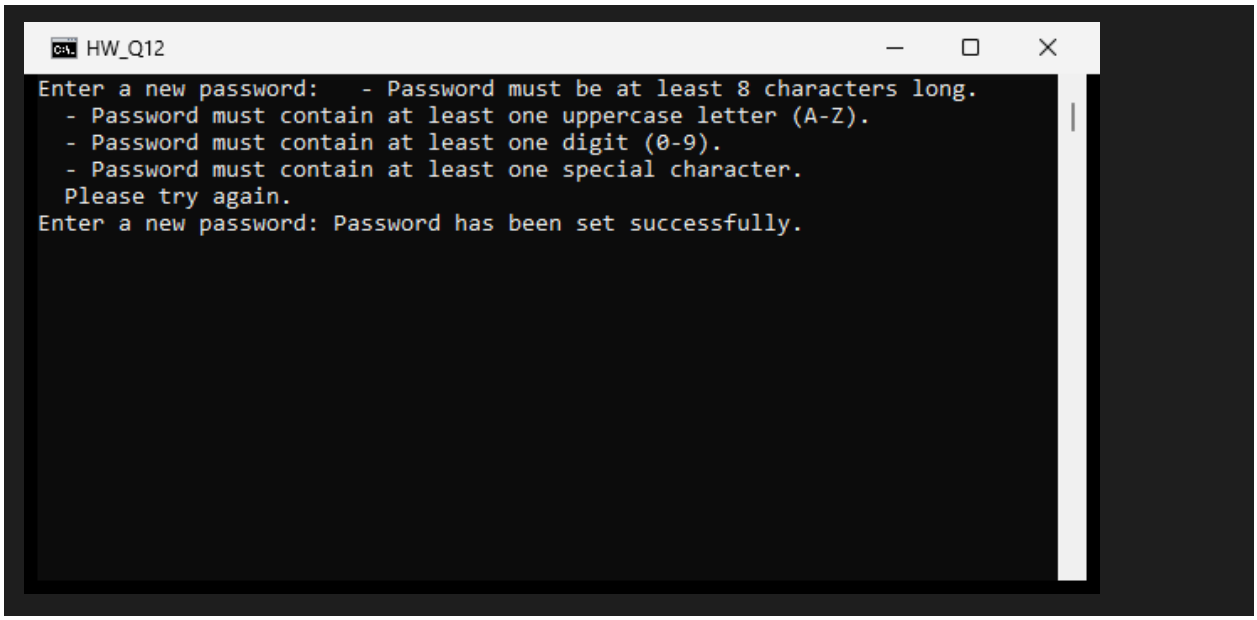

---

## Question 12: Secure Password Creator

```
/* Q12. Secure Password Creator:
       Repeatedly prompt for a password until it meets all criteria:
       - at least 8 characters
       - at least one uppercase letter (A-Z)
       - at least one lowercase letter (a-z)
       - at least one digit (0-9)
       - at least one special character
       For each failed attempt, list exactly which requirements were not met. */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main()
{
    char password[100];
    char special[] = "!@#$%^&*()_+-=[]{}|;:'\",.<>/?";
    int len, hasUpper, hasLower, hasDigit, hasSpecial;
    int valid, i, j;

    do
    {
        printf("Enter a new password: ");
        fgets(password, sizeof(password), stdin);
        password[strcspn(password, "\n")] = '\0';

        len = strlen(password);
        hasUpper = hasLower = hasDigit = hasSpecial = 0;

        for (i = 0; i < len; i++)
        {
            if (isupper(password[i]))
                hasUpper = 1;
            if (islower(password[i]))
                hasLower = 1;
            if (isdigit(password[i]))
                hasDigit = 1;
            for (j = 0; special[j] != '\0'; j++)
```

```
            {
                if (password[i] == special[j])
                    hasSpecial = 1;
            }
        }

        valid = 1;
        if (len < 8)
        {
            printf("  - Password must be at least 8 characters long.\n");
            valid = 0;
        }
        if (!hasUpper)
        {
            printf("  - Password must contain at least one uppercase letter (A-Z).\n");
            valid = 0;
        }
        if (!hasLower)
        {
            printf("  - Password must contain at least one lowercase letter (a-z).\n");
            valid = 0;
        }
        if (!hasDigit)
        {
            printf("  - Password must contain at least one digit (0-9).\n");
            valid = 0;
        }
        if (!hasSpecial)
        {
            printf("  - Password must contain at least one special character.\n");
            valid = 0;
        }

        if (!valid)
            printf("  Please try again.\n");
    } while (!valid);

    printf("Password has been set successfully.\n");

    return 0;
}
```

**Output:**




---

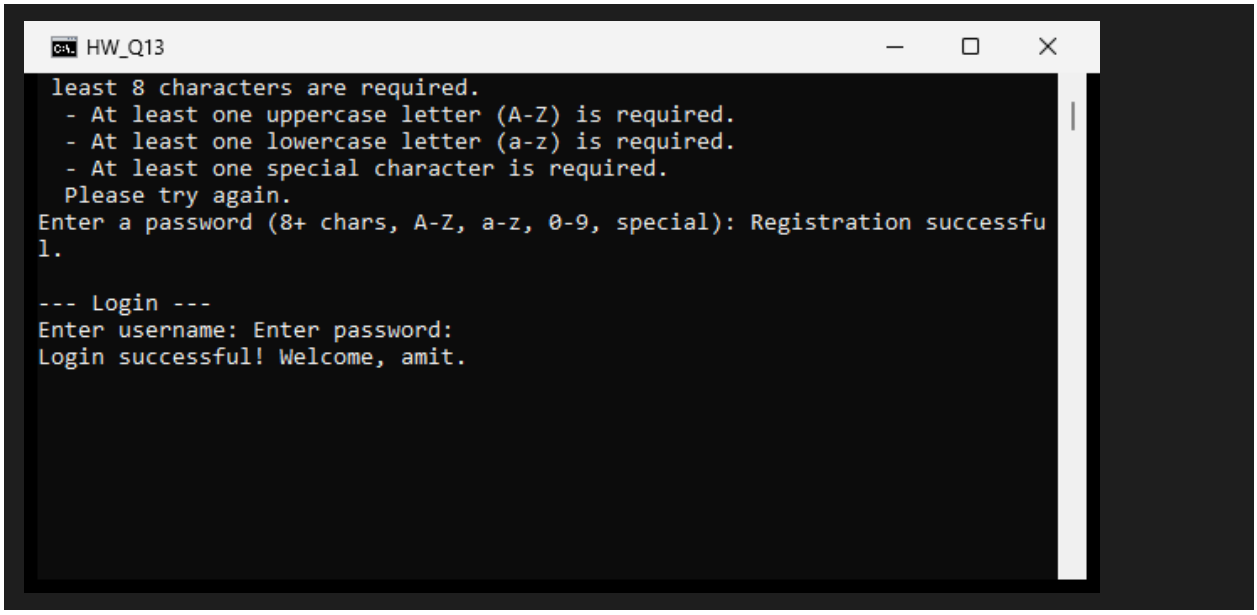
## Question 13: Secure User Authentication (registration + login)

```
/* Q13. Secure User Authentication:
       Registration: create a username and a password that meets security
       criteria (min 8 chars, at least one uppercase, one lowercase, one digit,
       one special character). Loop until the password is valid.
       Then login: validate entered credentials and display success/failure. */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int checkPassword(char p[])
{
    char special[] = "!@#$%^&*()_+-=[]{};:'\",.<>/?";
    int len = strlen(p);
    int hasUpper = 0, hasLower = 0, hasDigit = 0, hasSpecial = 0;
    int fail = 0, i, j;

    if (len < 8)
    {
        printf("  - At least 8 characters are required.\n");
        fail = 1;
    }

    for (i = 0; p[i] != '\0'; i++)
    {
        if (isupper(p[i]))
            hasUpper = 1;
        else if (islower(p[i]))
            hasLower = 1;
        else if (isdigit(p[i]))
            hasDigit = 1;
        else
        {
            for (j = 0; special[j] != '\0'; j++)
            {
                if (p[i] == special[j])
                    hasSpecial = 1;
            }
```

```
        }
    }

    if (!hasUpper)
    {
        printf("  - At least one uppercase letter (A-Z) is required.\n");
        fail = 1;
    }
    if (!hasLower)
    {
        printf("  - At least one lowercase letter (a-z) is required.\n");
        fail = 1;
    }
    if (!hasDigit)
    {
        printf("  - At least one digit (0-9) is required.\n");
        fail = 1;
    }
    if (!hasSpecial)
    {
        printf("  - At least one special character is required.\n");
        fail = 1;
    }

    return fail;
}

int main()
{
    char username[50], password[50];
    char loginUser[50], loginPass[50];
    char ch;
    int retry;

    printf("--- Registration ---\n");
    printf("Enter username: ");
    scanf("%49s", username);
    while ((ch = getchar()) != '\n' && ch != EOF)
        ;

    do
    {
        printf("Enter a password (8+ chars, A-Z, a-z, 0-9, special): ");
        fgets(password, sizeof(password), stdin);
        password[strcspn(password, "\n")] = '\0';

        retry = checkPassword(password);
        if (retry != 0)
            printf("  Please try again.\n");
    } while (retry != 0);

    printf("Registration successful.\n");

    printf("\n--- Login ---\n");
    printf("Enter username: ");
    scanf("%49s", loginUser);
    printf("Enter password: ");
    scanf("%49s", loginPass);

    if (strcmp(username, loginUser) == 0 && strcmp(password, loginPass) == 0)
        printf("\nLogin successful! Welcome, %s.\n", loginUser);
    else
        printf("\nLogin failed! Invalid username or password.\n");
```

```
    return 0;
}
```

**Output:**




---

## MCQ Questions: Answers

| Q | Answer |
|---|---|
| 1 | B) 8 |
| 2 | C) GoodDay |
| 3 | C) 0 |
| 4 | B) e |
| 5 | C) char str[] = "Hello"; |
| 6 | B) There |
| 7 | B) Mango |
| 8 | n |
| 9 | C) John |
| 10 | A) ProgrammingLanguage / Programming |

---

## Explain

**1. Why is the null character ('\0') important in a string? Explain with an example.**

In C, a string is just a character array with no separate length field, so the null character marks where the actual text ends. Every string-handling function (strlen, strcpy, strcmp, strcat, printf with %s, and so on) scans forward

until it hits '\0' and treats everything before that as the string's content; without it, these functions would keep reading past the intended data into whatever memory happens to follow, producing garbage output or a crash.

```
char str[6] = "Hello";  // stored as 'H','e','l','l','o','\0'
printf("%s", str);      // stops printing right before the '\0'
```

**2. Differentiate between the following string functions with suitable examples and how it works: strlen(), strcpy(), strcmp(), strcat()**

strlen(str) walks the string one character at a time and counts how many characters appear before the terminating '\0', returning that count. it does not include the null character itself.

strcpy(dest, src) copies the characters of src into dest one by one, including the terminating '\0', overwriting whatever was previously in dest. The destination array must be large enough to hold the source string.

strcmp(str1, str2) compares two strings character by character based on their ASCII values. It returns 0 if they are identical, a negative value if str1 comes before str2 alphabetically, and a positive value if str1 comes after str2.

strcat(dest, src) appends a copy of src onto the end of dest, starting at dest's existing null terminator, and adds a new '\0' after the combined content. dest must have enough spare space to hold both strings.

```
char a[20] = "Hello";
char b[] = "World";

printf("%lu\n", strlen(a));   /* 5 */
strcpy(a, b);                 /* a becomes "World" */
printf("%d\n", strcmp(a, b)); /* 0, since a and b are now equal */
strcat(a, "!!");               /* a becomes "World!!" */
```

**3. Explain the difference between comparing two strings using the == operator and using the strcmp() function.**

When applied to two arrays (or char pointers), the == operator compares their memory addresses, not their contents - it only returns true if both names refer to the exact same block of memory, which is almost never the case for two separately declared strings even when their text is identical. strcmp(), by contrast, walks through both strings character by character and compares the actual content, returning 0 when the text matches regardless of where each string lives in memory. For this reason, strcmp() (or an equivalent content comparison) is the correct way to check whether two C strings hold the same text.

**4. What is a two-dimensional character array (array of strings)? Explain its declaration and initialization with an example.**

A two-dimensional character array is a block of memory organized as a fixed number of rows, where each row is itself a character array capable of holding one string (including its terminating '\0'). It is a simple way to store a list of strings of similar maximum length using a single variable, and each row can be accessed and manipulated independently with the standard string functions.

```
char names[3][10] = {"Amit", "Rohan", "Priya"};

printf("%s\n", names[0]);  /* Amit  */
printf("%s\n", names[1]);  /* Rohan */
printf("%c\n", names[2][1]); /* 'r' - second character of "Priya" */
```

**5. Explain How Strings are Stored in a Two-Dimensional Character Array:**

**char cities[3][10] = {"Delhi", "Mumbai", "Chennai"};**

**What happens if a string longer than 9 characters is stored in one of the rows?**

Here cities are laid out as three separate rows placed back-to-back in memory, each reserved for exactly 10 bytes. "Delhi" occupies cities[0] (5 characters plus a null terminator, with unused bytes left over), "Mumbai" occupies cities[1], and "Chennai" occupies cities[2], each null-terminated within its own row.

Each row can safely hold at most 9 visible characters plus the null terminator, since the row size is 10. If a string longer than that is written into one row - for example, using strcpy(cities[0], "Constantinople") - the extra characters spill past the end of that row's 10 bytes and overwrite the beginning of the next row (or whatever memory follows). This is a buffer overflow: it is undefined behavior in C, and it can silently corrupt the next string in the array or other nearby data.